Hitch Talent Agent — Candidate Tile Generator

Project Overview

This is an internal tool for Hitch Partners, an executive recruiting firm specializing in cybersecurity roles. The tool generates client-facing Candidate Tile documents from Airtable data.

Candidate Tile — A one-page candidate profile PowerPoint with:
- Candidate bio (name, title, company, location, education, contact)
- AI-synthesized relevant security experience
- AI-synthesized current situation
- AI-synthesized anticipated concerns

Architecture

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                    AIRTABLE                                                  │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                              │
│  PEOPLE TABLE                                  CANDIDATE TILE TABLE                          │
│  ──────────────                                ────────────────────                          │
│  • Full Name                                   • Person (linked → People)                    │
│  • Current Title                               • Search (linked → Search)                    │
│  • Current Company                             • Lookup fields (from People)                 │
│  • Location                                    • Lookup fields (from Search)                 │
│  • Education                                   • Relevant Security Experience                │
│  • Email / Phone                               • Current Situation                           │
│  • Photo                                       • Anticipated Concerns                        │
│  • Resume (PDF)                                • Tile Draft Status                           │
│  • Notes                                       • Candidate Tile PowerPoint                   │
│                                                                                              │
│  SEARCH TABLE                                                                                │
│  ────────────                                                                                │
│  • Role Title                                                                                │
│  • Client                                                                                    │
│                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │ Airtable Button triggers webhook
                                  ▼
```

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                VERCEL SERVERLESS FUNCTIONS                                                   │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                                              │
│  POST /api/generate-tile-draft                                                                               │
│  ──────────────────────────────────────────                                                                  │
│  1. Receives tileId from Airtable webhook                                                                    │
│  2. Fetches Candidate Tile record (lookups auto-populated)                                                   │
│  3. Downloads and parses Resume PDF (if present)                                                             │
│  4. Calls Claude API to synthesize content                                                                   │
│  5. Writes draft content back to Airtable                                                                    │
│  6. Sets status to "Draft Ready"                                                                             │
│                                                                                                              │
│                                                                                                              │
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  │  PM reviews/edits content directly in Airtable                                                            │
│  │  Sets status to "Approved" when satisfied                                                                 │
│                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│                                                                                                              │
│  POST /api/generate-tile-pptx                                                                                │
│  ──────────────────────────────────────────                                                                  │
│  1. Receives tileId from Airtable webhook                                                                    │
│  2. Validates status = "Approved"                                                                            │
│  3. Fetches approved content from Airtable                                                                   │
│  4. Generates PowerPoint with pptxgenjs                                                                      │
│  5. Uploads to Vercel Blob (public URL)                                                                      │
│  6. Updates Airtable attachment field                                                                        │
│                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Workflow

1. **PM creates Candidate Tile record** in Airtable, linking to a Person and Search
2. **PM clicks "Generate Draft" button** → Triggers webhook to `/api/generate-tile-draft`
3. **AI synthesizes content** from resume and notes → Writes to Airtable fields
4. **PM reviews and edits** the three content fields directly in Airtable
5. **PM sets status to "Approved"** when satisfied

6. **PM clicks "Generate PowerPoint" button** → Triggers webhook to `/api/generate-tile-pptx`
7. **System generates PPTX** and uploads to Airtable attachment field
8. **PM downloads and shares** the Candidate Tile with client

Tech Stack

Component	**Technology**	**Purpose**
Runtime	Node.js 20 (Vercel)	Serverless functions
Database	Airtable	Source of truth for all data
AI	Anthropic Claude API (Haiku)	Content synthesis
PPTX Generation	pptxgenjs	PowerPoint file creation
PDF Parsing	pdf-parse	Resume text extraction
File Hosting	Vercel Blob	Public URLs for PPTX attachments

Airtable Schema

People Table

Field	**Type**	**Purpose**
Full Name	Single line text	Candidate's full name
Current Title	Single line text	Current job title
Current Company	Single line text	Current employer
Location	Single line text	City, State or City, Country
Education	Single line text	Degree and institution
Email	Email	Contact email
Phone	Phone	Contact phone
Photo	Attachment	Headshot image
Resume	Attachment	PDF resume
Notes	Long text (rich)	Recruiter notes from calls/screening

Search Table

Field	**Type**	**Purpose**
Name	Single line text	Search/project identifier
Role Title	Single line text	e.g., "Chief Information Security Officer"
Client	Single line text	Client company name

Candidate Tile Table

Field	**Type**	**Purpose**

Person	Linked record	Links to People table (single)
Search	Linked record	Links to Search table (single)
Candidate Name	Lookup	From People → Full Name
Current Title	Lookup	From People → Current Title
Current Company	Lookup	From People → Current Company
Location	Lookup	From People → Location
Education	Lookup	From People → Education
Email	Lookup	From People → Email
Phone	Lookup	From People → Phone
Photo	Lookup	From People → Photo
Resume	Lookup	From People → Resume
Notes	Lookup	From People → Notes
Role Title	Lookup	From Search → Role Title
Client	Lookup	From Search → Client
Relevant Security Experience	Long text (rich)	AI-generated, PM-editable
Current Situation	Long text (rich)	AI-generated, PM-editable
Anticipated Concerns	Long text (rich)	AI-generated, PM-editable
Tile Draft Status	Single select	"Not Started" / "Draft Ready" / "Approved" / "Draft Error"
Tile Draft Error	Long text	Error message if draft generation failed
Resume Parse Status	Single select	"Not Attempted" / "Success" / "Failed" / "No Resume"
Tile Generation Count	Number	Tracks regeneration attempts (default: 0)
Candidate Tile PowerPoint	Attachment	Final generated document

Tile Draft Status options:
- `Not Started`
- `Draft Ready`
- `Approved`
- `Draft Error`

Resume Parse Status options:
- `Not Attempted`
- `Success`
- `Failed`
- `No Resume`

Environment Variables
```bash
# Airtable
AIRTABLE_API_KEY=pat_xxxxxxxxxxxx          # Personal access token
AIRTABLE_BASE_ID=appXXXXXXXXXXXXXX         # Base ID from URL

# Anthropic
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxx      # API key
```

```
# Vercel Blob (auto-configured in Vercel)
BLOB_READ_WRITE_TOKEN=vercel_blob_xxxxx    # Added automatically when you enable Blob

# Internal authentication
INTERNAL_API_KEY=your-secret-key-here      # For endpoint authentication

# Assets
HITCH_LOGO_URL=https://...                 # Publicly accessible Hitch Partners logo URL
```

File Structure

```
hitch-talent-agent/
├── CLAUDE.md                  # This file
├── README.md                  # Setup and deployment instructions
├── .env.example               # Template for environment variables
├── .gitignore                 # Excludes .env*, node_modules, etc.
├── package.json
├── vercel.json
│
├── api/
│   ├── generate-tile-draft.js    # POST: Generate candidate tile draft
│   └── generate-tile-pptx.js     # POST: Generate candidate tile PowerPoint
│
├── lib/
│   ├── airtable.js            # Airtable API client with helpers
│   ├── anthropic.js           # Claude API wrapper
│   ├── pptx-tile.js           # Candidate Tile PowerPoint generation
│   ├── pdf-extract.js         # Resume text extraction
│   └── logger.js              # Structured logging utility
│
└── templates/
    └── (reserved for future template files)
```

API Endpoint Specifications

POST /api/generate-tile-draft

Input:
```json
{
  "tileId": "recXXXXXXXXXXXXXX"
```

}
```

**Process:**
1. Authenticate request (check `x-api-key` header)
2. Log: `request_received`
3. Fetch Candidate Tile record by ID (lookups auto-populated by Airtable)
4. Validate: Record exists
5. Validate: Has linked Person (Candidate Name lookup is not empty)
6. Validate: Status is not "Approved" (idempotency check)
7. Check generation count — reject if >= 5
8. Extract candidate info using `getFieldValue` helper:
   - Candidate Name, Current Title, Current Company
   - Role Title, Client (from Search lookup)
   - Notes
9. Check for Resume attachment:
   - If exists: Download and extract text
     - On success: Set `Resume Parse Status` = "Success"
     - On failure: Log warning, set `Resume Parse Status` = "Failed", continue without resume
   - If no resume: Set `Resume Parse Status` = "No Resume"
10. Log: `airtable_fetch_complete`
11. Call Claude API to synthesize:
    - Relevant Security Experience (2-4 sentences)
    - Current Situation (1-2 sentences)
    - Anticipated Concerns (2-3 bullet points)
12. Log: `claude_api_complete`
13. Update Airtable:
    - `Relevant Security Experience`: Claude output
    - `Current Situation`: Claude output
    - `Anticipated Concerns`: Claude output
    - `Tile Draft Status`: "Draft Ready"
    - `Resume Parse Status`: (as determined above)
    - `Tile Generation Count`: increment by 1
14. Log: `airtable_updated`
15. Return success response with any warnings

**Success Response:**
```json
{
  "status": "success",
  "message": "Candidate tile draft generated",
  "data": {
    "tileId": "recXXXXXXXXXXXXXX",
    "candidateName": "Jane Doe",

```
    "generationCount": 1
  },
  "warnings": []
}
```

Error Response:
```json
{
  "status": "error",
  "message": "Candidate Tile must be linked to a Person record",
  "data": null,
  "warnings": []
}
```

Partial Success Response (resume parsing failed):
```json
{
  "status": "success",
  "message": "Candidate tile draft generated",
  "data": {
    "tileId": "recXXXXXXXXXXXXXX",
    "candidateName": "Jane Doe",
    "generationCount": 1
  },
  "warnings": ["Resume could not be parsed: Invalid PDF format"]
}
```

POST /api/generate-tile-pptx

Input:
```json
{
  "tileId": "recXXXXXXXXXXXXXX"
}
```

Process:
1. Authenticate request (check `x-api-key` header)
2. Log: `request_received`
3. Fetch Candidate Tile record by ID
4. Validate: `Tile Draft Status` = "Approved" (reject otherwise)

5. Extract all fields using `getFieldValue` helper
6. Get Photo attachment URL (if present)
7. Get Hitch Logo URL from environment variable
8. Log: `airtable_fetch_complete`
9. Call `createCandidateTilePresentation()` to generate PPTX buffer
10. Log: `pptx_generated`
11. Upload buffer to Vercel Blob:
```javascript
    const { url } = await put(
      `tiles/${tileId}-${Date.now()}.pptx`,
      pptxBuffer,
      {
        access: 'public',
        contentType: 'application/vnd.openxmlformats-officedocument.presentationml.presentation'
      }
    );
```
12. Log: `blob_uploaded`
13. Update Airtable: `Candidate Tile PowerPoint` attachment field with Blob URL
14. Log: `airtable_updated`
15. Return success response

Success Response:
```json
{
  "status": "success",
  "message": "Candidate tile PowerPoint generated",
  "data": {
    "tileId": "recXXXXXXXXXXXXXX",
    "candidateName": "Jane Doe",
    "pptxUrl": "https://blob.vercel-storage.com/tiles/rec...-123456.pptx"
  },
  "warnings": []
}
```

Error Response (not approved):
```json
{
  "status": "error",
  "message": "Cannot generate PowerPoint: draft status is 'Draft Ready', must be 'Approved'",
  "data": null,
  "warnings": []
}
```

```

## Library Specifications

### lib/airtable.js
```javascript
// Core functions
async function getRecord(table, recordId)
// Fetches a single record by ID
// Returns: { id, fields, createdTime }

async function updateRecord(table, recordId, fields)
// Updates a record with new field values
// Returns: { id, fields, createdTime }

// Helper functions
function getFieldValue(fields, fieldName, defaultValue = "")
// Safely extracts value from lookup fields (which return arrays)
// Usage: getFieldValue(fields, "Candidate Name", "Unknown")
// Returns: string (first array element or defaultValue)

function getAttachmentUrl(fields, fieldName)
// Returns first attachment URL or null
// Usage: getAttachmentUrl(fields, "Resume")

// Rate limiting
// Implement exponential backoff: wait 1s, 2s, 4s on 429 responses
// Max 3 retries before failing
```

lib/anthropic.js
```javascript
const MODEL = "claude-haiku-4-5-20251001";  // Using Haiku for cost efficiency
const MAX_TOKENS_SYNTHESIS = 800;

async function synthesizeCandidateContent(candidateData, roleContext, resumeText, notes)
// Input:
//   candidateData: { name, title, company }
//   roleContext: { roleTitle, clientName }
//   resumeText: string (may be empty)
//   notes: string (may be empty)
// Returns: {
//   relevantExperience: string,    // 2-4 sentences
//   currentSituation: string,      // 1-2 sentences
```
```

```
//   anticipatedConcerns: string    // 2-3 bullet points
// }
```

Prompt Template:
```
You are a senior executive recruiter preparing a candidate brief for a {roleTitle} position at
{clientName}.

CANDIDATE: {name}
CURRENT ROLE: {title} at {company}

RESUME CONTENT:
{resumeText or "No resume available"}

RECRUITER NOTES:
{notes or "No notes available"}

Generate three sections for the candidate tile. Be specific and reference actual experience from
the resume and notes. Avoid generic language.

1. RELEVANT SECURITY EXPERIENCE (2-4 sentences)
Focus on experience directly relevant to this {roleTitle} role. Highlight specific security domains,
company types, team sizes, or notable achievements.

2. CURRENT SITUATION (1-2 sentences)
Why are they open to this opportunity? What are they looking for? Include timing if known.

3. ANTICIPATED CONCERNS (2-3 bullet points, brief)
What objections might the client have? What risks should we address proactively? Consider:
compensation expectations, relocation requirements, experience gaps, flight risk, cultural fit.

Respond in this exact JSON format:
{
   "relevantExperience": "...",
   "currentSituation": "...",
   "anticipatedConcerns": "• ...\\n• ...\\n• ..."
}
```

lib/pptx-tile.js
```javascript
async function createCandidateTilePresentation(data)
// Input: {
```

```
//   candidateName: string,
//   currentTitle: string,
//   currentCompany: string,
//   location: string,
//   education: string,
//   email: string,
//   phone: string,
//   relevantExperience: string,
//   currentSituation: string,
//   anticipatedConcerns: string,
//   roleTitle: string,
//   clientName: string,
//   photoUrl: string | null,
//   hitchLogoUrl: string | null
// }
// Output: Buffer (PPTX file contents)
```

lib/pdf-extract.js

```javascript
async function extractTextFromPdf(url)
// Downloads PDF from URL, extracts text using pdf-parse
// Truncates to first 8000 characters to avoid huge prompts
// Returns: {
//   success: boolean,
//   text: string,
//   error: string | null
// }
// On failure: { success: false, text: "", error: "Error message" }
```

lib/logger.js

```javascript
function log(event, data = {})
// Outputs structured JSON log with timestamp
// Usage: log('request_received', { endpoint: 'generate-tile-draft', tileId })

// Standard events for Candidate Tile:
// - request_received
// - airtable_fetch_complete
// - pdf_parse_complete
// - pdf_parse_failed
// - claude_api_called
// - claude_api_complete
```

// - pptx_generated
// - blob_uploaded
// - airtable_updated
// - error
```

## PowerPoint Layout Specification

### Candidate Tile Slide

**Dimensions:** 13.333" x 7.5" (16:9 widescreen)

**Color Palette:**

| **Use** | **Hex** | **RGB** |
|-----|-----|-----|
| Navigation (headers, primary text) | `#1B365D` | 27, 54, 93 |
| White (background) | `#FFFFFF` | 255, 255, 255 |
| Slate (secondary text) | `#64748B` | 100, 116, 139 |
| Accent (dividers) | `#0EA5E9` | 14, 165, 233 |
| Placeholder (no photo) | `#D4D4D8` | 212, 212, 216 |

**Layout:**
```
┌──────────────

```
│  │          │                                  │
│  └──────────────────┘  📍 {Location}                          │
│                 🎓 {Education}                          │
│  If no photo:     ✉️ {Email} | 📱 {Phone}                    │
│  Gray box #D4D4D8  x: 2.2", y: 3.0", font: 10pt, #64748B         │
│                                                  │
├──────────────────
```

generate-tile-draft	Generation count >= 5	"Generation limit reached (5). Contact admin to reset."
generate-tile-pptx	Candidate Tile not found	"Candidate Tile not found: {tileId}"
generate-tile-pptx	Status ≠ "Approved"	"Cannot generate PowerPoint: draft status is '{status}', must be 'Approved'"

Recoverable Errors (continue with degraded functionality)

Error	**Handling**	**Field Update**
Resume PDF parse failure	Continue without resume text, log warning	`Resume Parse Status` = "Failed"
Photo attachment missing	Render gray placeholder box	(none)
Claude API timeout	Retry once, then fail	`Tile Draft Status` = "Draft Error"

Rate Limiting

Airtable allows 5 requests/second per base. Implement exponential backoff:
- On 429 response: wait 1 second, retry
- Second 429: wait 2 seconds, retry
- Third 429: wait 4 seconds, retry
- Fourth 429: fail with error

Error Response Structure

All endpoints return consistent error format:
```json
{
  "status": "error",
  "message": "Human-readable error description",
  "data": null,
  "warnings": []
}
```

Idempotency Rules

Generate Draft Button

Current Status	**Action**
"Not Started"	Proceed with generation
"Draft Ready"	Proceed (overwrites previous draft)
"Approved"	**Reject** — return error

| "Draft Error" | Proceed with generation |

Generate PPTX Button

Current Status	**Action**
"Approved"	Proceed — always safe to regenerate PPTX
Any other status	**Reject** — return error

Security

Authentication

All endpoints require `x-api-key` header matching `INTERNAL_API_KEY` environment variable:
```javascript
export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({
      status: "error",
      message: "Method not allowed",
      data: null,
      warnings: []
    });
  }

  const apiKey = req.headers['x-api-key'];
  if (apiKey !== process.env.INTERNAL_API_KEY) {
    return res.status(401).json({
      status: "error",
      message: "Unauthorized",
      data: null,
      warnings: []
    });
  }

  // ... rest of handler
}
```

Airtable Token Scopes

Create Personal Access Token with minimum required scopes:
- `data.records:read`
- `data.records:write`

- `schema.bases:read`

Grant access only to the specific base, not "All current and future bases."

Environment Variable Management

1. Never commit `.env` files — `.gitignore` must include `.env*`
2. Add Vercel environment variables via dashboard or CLI:
```bash
  vercel env add AIRTABLE_API_KEY production
  vercel env add ANTHROPIC_API_KEY production
  vercel env add INTERNAL_API_KEY production
```
3. Pull to local for development: `vercel env pull .env.local`

Logging

Use structured JSON logging for all events:
```javascript
import { log } from '../lib/logger.js';

log('request_received', { endpoint: 'generate-tile-draft', tileId });
log('airtable_fetch_complete', { candidateName: 'Jane Doe' });
log('pdf_parse_complete', { characterCount: 5432 });
log('claude_api_called', { model: 'claude-haiku-4-5-20251001' });
log('claude_api_complete', { completionTokens: 342 });
log('pptx_generated', { fileSize: 45632 });
log('blob_uploaded', { url: 'https://...' });
log('airtable_updated', { field: 'Candidate Tile PowerPoint' });
log('error', { error: error.message, stack: error.stack });
```

Commands
```bash
# Development
npm install                 # Install dependencies
vercel dev                  # Run locally on http://localhost:3000
vercel env pull .env.local    # Pull env vars for local dev

# Deployment
vercel deploy               # Deploy preview
vercel deploy --prod        # Deploy production

# Testing
```

```
curl -X POST http://localhost:3000/api/generate-tile-draft \
  -H "Content-Type: application/json" \
  -H "x-api-key: your-secret-key" \
  -d '{"tileId": "recXXXXXXXXXXXXXX"}'

curl -X POST http://localhost:3000/api/generate-tile-pptx \
  -H "Content-Type: application/json" \
  -H "x-api-key: your-secret-key" \
  -d '{"tileId": "recXXXXXXXXXXXXXX"}'
```